Computer Graphics: Lecture 16

Lighting part 2: Phong and Goureaud shading

Slide Deck © Grant Williams, 2026, License: CC-BY-SA 4.0

Welcome back!

Last time we learned about diffuse (Lambertian) shading!

And it was cool, we saw some nice simple shading applied to our models.

But they were a little drab. We couldn't make them shiny.

That changes today…

The teapot will *shine*.

This time

We're going to make several improvements to our lighting code:

- We're going to make it capable of shininess
- We're going to allow lights to have a position in space, so that getting farther away makes them dimmer (attenuation).
- We're going to allow *multiple lights* at one time.
- We're going to improve our shader and the software design of our sample to make bind-group management nicer. This means using more structs in our shader.
- We're going to learn about struct alignment in WebGPU.
  (I couldn't put it off anymore 😔)

This time (2)

The concepts in this lecture are rather challenging, and in the past I've noticed that we often needed to spend some more time here.

So let's do that. This might be a 2-parter!

Revisiting lambertian lighting

So what's missing from lambertian lighting?

We can clearly see the light on the teapot.

And yet, I've never seen a teapot that looks so…matte before.

Like one of those sports car paintjobs that don't shine at all.



Specular highlights are missing

You know what that sports car doesn't do? Reflect the sun directly into your eyeball when you're eating at a restaurant.

[Has this ever happened to you?]

Those little bright reflections are called specular highlights. They are caused by the photons getting reflected off the surface *right into your eye*. Not getting scattered equally in all directions.

They don't have to be annoying like the sun reflecting off a car. They can give a metallic object its shine. Suffice to say: you're used to seeing them, and not having them is making the mesh look unrealistic!

Specular highlight example



Lambertian-lit teapot.



Teapot with specular highlights.

How do they work

No one is going to confuse either teapot for the real thing, but clearly we've taken another step toward realism.

Objects without visible specular highlights are curiousities. Even a rock or a tree stump will have some "shine" to it.

But in order to include specular highlights in our shader, we're going to need a more advanced illumination model…

The Phong model

The next model we're going to learn is called **The Phong Reflection** model. It's named after the Bùi Tường Phong[1], a highly influential computer graphics researcher from the university of Utah. The same place that gave us the teapot.

He sadly passed away very young. However, the reflection model he developed is one of the most visually iconic things from early 3D graphics, and it is still widely used today.

Just to drive this home before we learn it…

[1] In Vietnam, the family name comes first, so Phong is actually his given name. It should probably have been named the Bùi model.

The Phong model's importance

The Phong model is found in:

- Used by the Sony Playstation, Nintendo 64, And Atari Jaguar, as well as pretty much every 3D console after that. (in the form of Blinn-Phong, Gouraud shading)
- Natively supported by pretty much every 3D modelling software, such as Blender, Maya, and 3DS Max.
- Lots of 80′s and 90′s era pre-rendered images, such as in this Stanford competition which goes back to 1992.
- Movies, too

It gives a distinctive look that accompanies a lot of 90′s CG. You'll start to notice it after we learn about it.

From Lambert to Phong

In our Lambertian shader, there were two components to the light:

- Ambient light, which represented the light that was being scattered throughout the scene.
- Diffuse light, for light that was scattered from the surface.

The Phong reflection model has *three* components:

- Ambient (same as above)
- Diffuse (same as above)
- Specular (new)

These components are added together, for each light, just like before.

Specular calculation

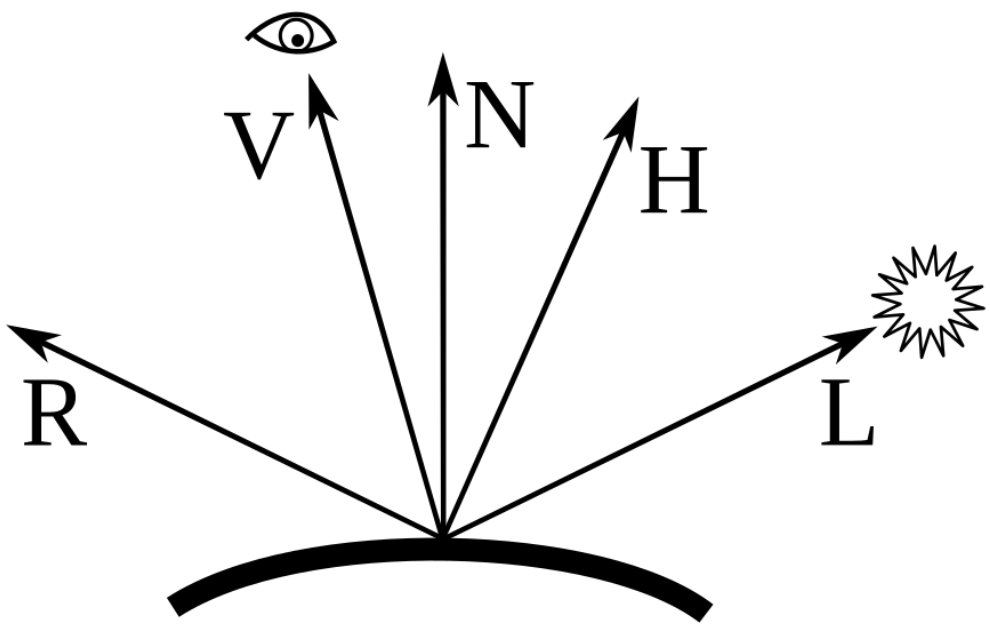
Recall that the diffuse component depended on the cosine between normal of the surface and the direction to the light.

The specular component is a bit more complicated:

- It definitely has to depend on the normal and the direction of the light like before…
- …However, it represents how light reflects into the eye. So it *also depends on where the eye is.*

Let's see an illustration.

Phong vectors



L is the vector pointing toward the light. N is the normal vector. V is the vector pointing towards the “eye” (camera). R is the reflection of L. Image by Martin Krause, Public Domain

Phong vectors (2)

Ignore the H vector for now. We'll talk about it later.

The other vectors are all important:

- We need a normal vector like before to tell us how the surface faces.
- This time, the way we use L and N is different. We *reflect* L over N.
- The amount of light that gets reflected is based on the cosine between *the eye and the reflected light.*
- Finally, a *shininess exponent* is applied so falloff is more or less steep.

The result of this calculation is called the **specular component**, but it's only one of three components in the Phong model

The Phong model

The Phong model is a way of calculating light. It's different from **Phong Shading**, which specifically means to use the Phong model in the fragment shader (as opposed to the vertex shader–which is called **Gouraud Shading**).

This is the high level view of the Phong model:

$$\text{ambient color} + \sum_{\text{light}} [\text{diffuse color} + (\text{specular color})^{\text{shininess}}]$$

We've already seen how to compute ambient and diffuse. Specular and shininess are new.

The Phong model (2)

In addition, the colors in the model have two factors:

- The color/brightness[1] of the light
- The color response of the **material**

We briefly saw materials last lecture, where we had ambient and diffuse response factors that represented how the mesh reacted.

Basically, the **material** of your 3D object is where we store its response to different kinds of light. We also often store the texture alongside the material. It describes how the model looks under illumination.

[1] We don't really distinguish between the color of the light and its brightness in the Phong model. Brighter lights have higher RGB values.

The Phong material properties

In order to use the Phong model, we have to describe how the object behaves under the 3 different lighting components, as well as its shininess value:

- Ambient: how much ambient light the object reflects
- Diffuse: how much diffuse light the object reflects
- Specular: how much specular light the object reflects
- Shininess: how small and focused the specular highlight is

These can be 4 scalar values, or the first 3 can be RGB values (which I've demonstrated in the sample). Shininess is always a scalar.

The basic model

Since the Phong model is additive, we can think of each kind of brightness as being a separate term. We just add them together:

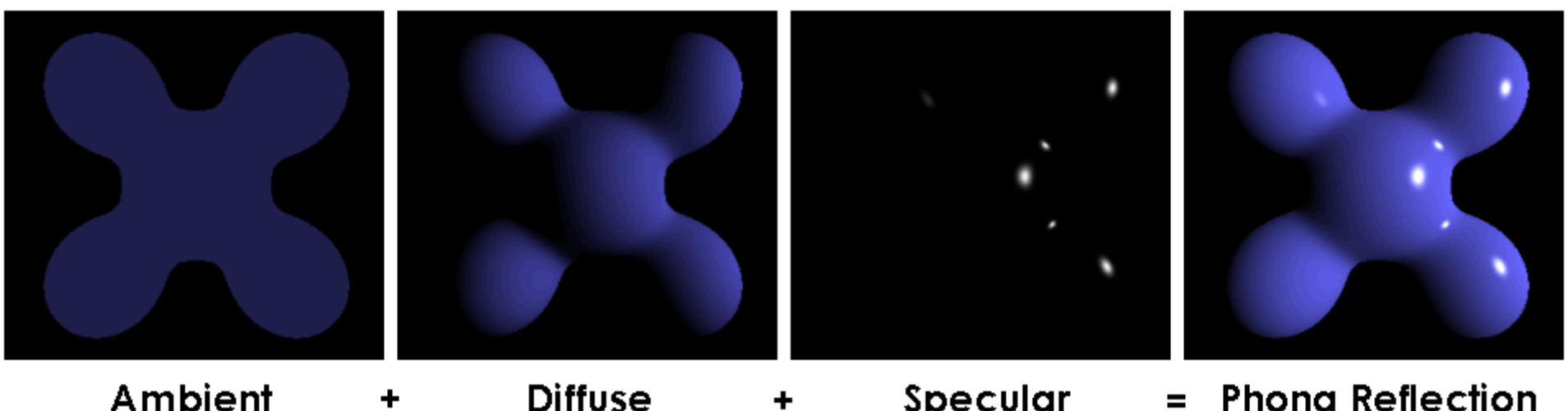


Image by Brad Smith, CC-BY-SA 3.0

The Phong Equation

The Phong Model is defined by its equation, from which we obtain the color to draw the pixel:

$$I_p = k_a i_a + \sum_{m \in \text{ lights}} \left(k_d \left(\hat{L}_m \cdot \hat{N} \right) i_{m,d} + k_s \left(\hat{R}_m \cdot \hat{V} \right)^{\alpha} i_{m,s} \right)$$

Here are what the variables mean:

- I_p the result we want to compute: either the color of a pixel or its brightness. It will be a brightness if we use scalars for all the lowercase variables, it will be a color if we use 3-vectors and pointwise multiplication.
- The “k” variables are the material properties: ambient, diffuse, and specular respectively.

The Phong Equation (2)

- The “m” subscript describes which light we’re talking about. If there are 8-lights, you will compute their diffuse and specular terms, and add the colors or brightnesses together (the big sigma).
- The L-hat is a vector from the surface to the light. N-hat is the normal
- R-hat is the light vector reflected over the normal.
- Alpha is “shininess”.
- The “i” factors are “illuminance”. Either color or brightness. The subscript tells us which light the factor is for, and which intensity[1]

[1] I left this out of my sample, because it’s rather abstract, but Phong is an empirical model, and being able to tune the specular “output” of a light may be useful even if it doesn’t make physical sense (specularness is a property of a surface, not a light)

Using the Phong Equation (3)

The way we use this equation is by making all those parameters (except "i" if you want) available to the shader as uniforms.

Then we program it into the shader. I'll show you how I did it in a bit.

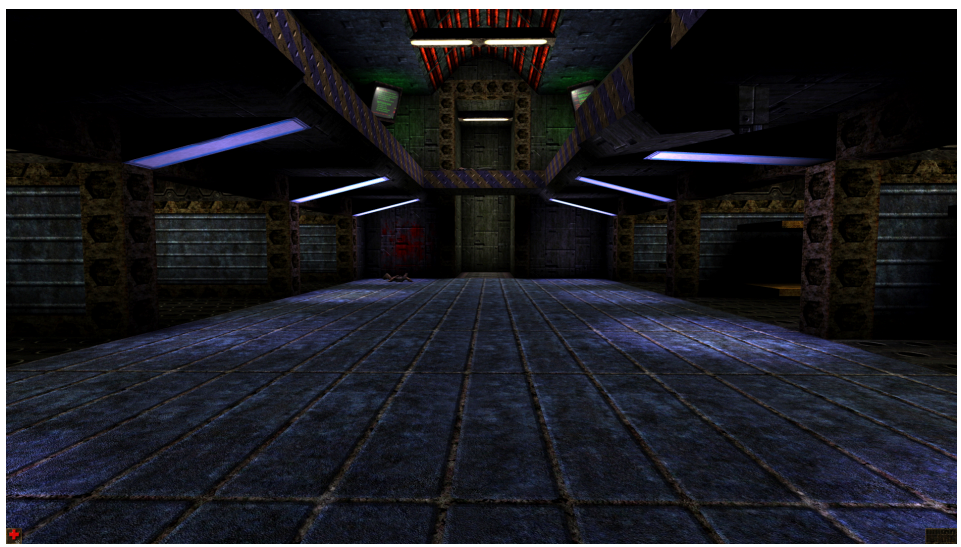
Artistically: when we're building our scene, not only do we decide how to place our meshes and which textures to use, we also give our 3D models material properties. For each model, we decide how much red, green, and blue its specular highlight reflects, how much diffuse and ambient light it reflects, and how small the highlights are. These are all determined in the model's material, and one material can be shared by many models. It's often combined with a texture, too.

Using the Phong Equation (4)

If the light value is a color vector, the result is the light color. If it's a scalar, it represents brightness, and the result is the light's brightness.



Quake (1996) did not support colored lighting. Its lighting values were just "brightnesses".

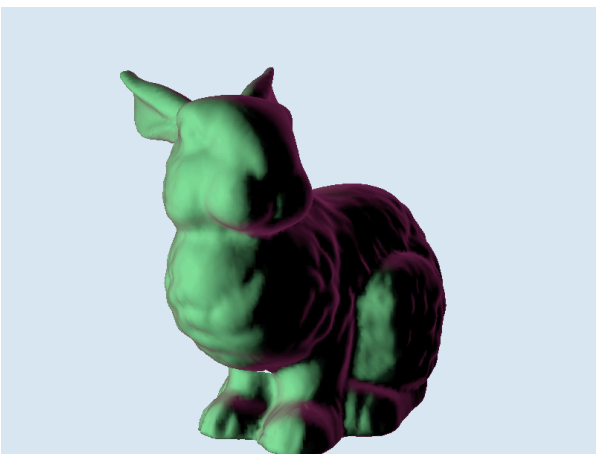


Unreal (1998) supported full colored lighting, which was a big deal at the time.

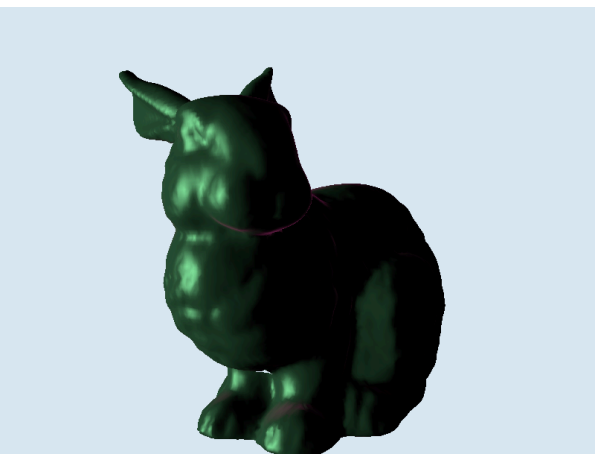
Questions?

What shininess does

One thing you might be wondering is what shininess does:



Shininess = 1.25 (ignore the pinkish light, we'll talk about it later)



Shininess =15



Shininess = 50

What shininess does (2)

Higher shininess makes the size of the specular spotlight *smaller*.

The result is usually that the surface looks more metallic.

The lights that we're simulating are infinitely small, so as the surface becomes more shiny, it's like you can see that tiny light in the reflection.

Mathmatically, remember that we're taking the dot-product between the reflected light and the eye. These are both unit vectors: the result will always be between -1 and 1 (and we make sure it's at least 0 to avoid "negative light"). Taking that number to a high power will make it need to be very close to 1 (direct reflection) to avoid falling off.

What shininess does (3)

Ironically, this means that increasing shininess means our model is less…well…shiny. Because there will be less specular color.

Typically, we can adjust for this by making the highlights more intense, by cranking up the material properties for specular.

One thing that distinguishes Phong from more modern, physically-based illumination models is that it is not actually modelling things like conservation of energy. It is absolutely possible to make the material reflection "reflect" more light than was emitted.[1]

---

[1] I didn't do this in the sample, but you can open the page inspector in your browser and force the controls past 1 for the RGB sliders.

Where Phong?

Now, we're ready to start using the Phong illumination model.

However, we are immediately faced with a decision: *where* to start using the Phong illumination model.

You see, there are actually several versions of Phong. Two versions use the same model, but are in different places…

- Using the Phong model in the fragment shader is called, straightforwardly, **Phong Shading**.
- Using the Phong model in the vertex shader is called **Gouraud Shading**.

Gouraud Shading

Named after *another* University of Utah Ph.D student, Henri Gouraud. Gouraud is French, but in English, the closest approximation of his last name is probably “goo-roh” (not “guh-rahd”).

Gouraud shading in when we do (usually) Phong shading at each vertex, and then, linearly interpolate between vertices.

This might be familiar: it’s what the GPU already does for all return values of the fragment shader: it interpolates them over the fragments.

So: we compute the Phong color in the vertex shader and return it as a vertex output. In the fragment shader we use that as our base color (maybe also multiplying it by a texture sample).

Gouraud Shading (2)

Phong shading generates smooth highlights, whereas Gouraud shading generates highlights that get dimmer as they fall away from vertices:

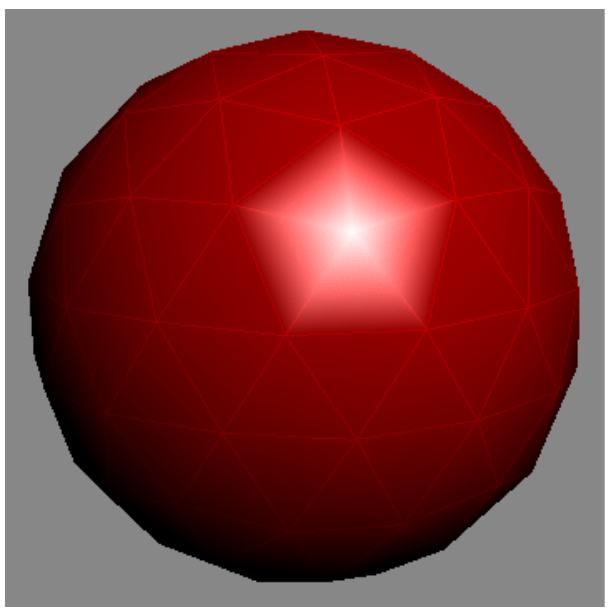


Image by Zom-B, CC-BY 2.0

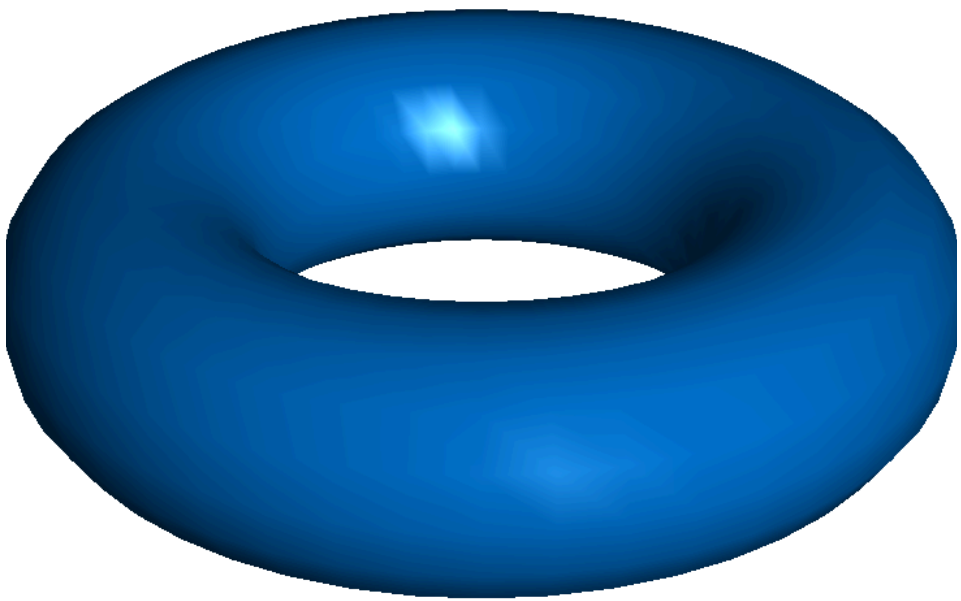


Image by Maarten Everts, public domain.

Gouraud Shading (3)

What's up with that weird artifacting?

It happens like this:

- We calculate the phong brightness/color in the vertex shader
- We return it from the vertex shader
- The fragment shader takes that color and either returns it directly, or multiplies it by the texture sample if texturing is being used.

So we're *linearly interpolating* the color between vertices.

This means the highlight will fall off a linear amount, which makes it look like a polygon.

Gouraud vs. Phong Shading



Gouraud shaded teapot. Notice the artifacting around the highlight.



Phong shaded teapot. The highlight is smooth (ignore the seams, those are part of the model)

Gouraud vs. Phong shading

With Phong shading, we interpolate the normal, rather than the color.

If that's all we did, Phong shading would have artifacts too, because linearly interpolating between two normal vectors makes the intermediate "normal" shorter.

But we *re-normalize* the normal in the phong shader. This makes the interpolation spherical instead of linear: we end up rotating the normal over the surface.

As a result, there isn't a linear falloff, and we don't get the artifacts.

Why Gouraud Shading?

So given that Gouraud shading has those artifacts, you might wonder why we use it at all?

It's mainly historical: gouraud shading was a generic technique for interpolating color over vertices. It was built in to early GPUs.

Adding the gouraud-shaded phong model added an extra calculation at those vertices, but that was all. It was tractable on those electronics.

Adding full Phong shading would have been much more expensive: it would have required the light value be recalculated at each pixel during interpolation. So Gouraud shading was considered good enough.

Blinn-Phong

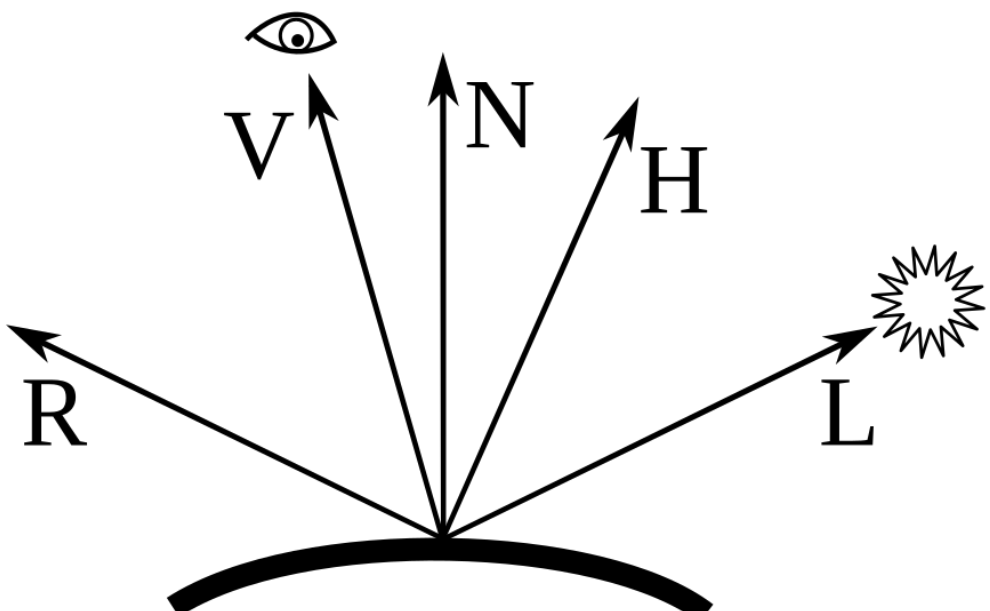
However, there is one more optimization that really made Gouraud-shading tractable, and which allowed it to use the same mechanism as the Lambertian diffuse shading.

It's called the Blinn-Phong model, named after *yet another* University of Utah researcher, Jim Blinn.

Blinn recognized that it was possible to approximate the Phong calculation used in Gouraud shading in a way that did not require computing the reflection vector.

Now we can finally appreciate the **H** vector from the diagram before...

Blinn-Phong (2)



The H vector is “halfway” between the light and the camera. That is, it is the average of the two. Image by Martin Krause, Public Domain

Blinn-Phong (3)

That H vector is the key. It is halfway been the direction to the light and the direction to the camera.

The way we generate this vector is by adding the camera and light vectors together and then normalizing.

Then, we take the cosine between this halfway vector and the normal vector. So the computation ends up being similar to diffuse shading.

The result is still raised to the power of shininess, although we usually need to make shininess 4-times higher than normal Phong for similar highlights (materials can have a separate Blinn shininess for this).

Blinn-Phong (4)

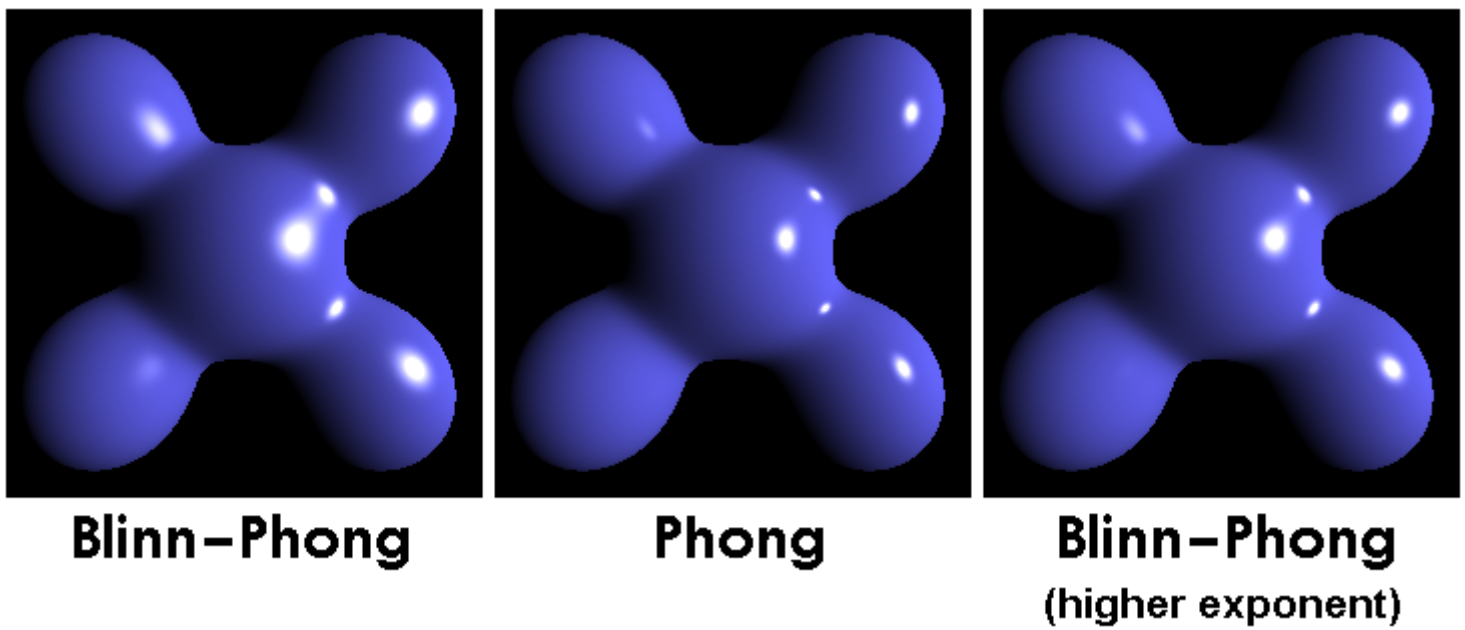
It may seem strange that the relationship between this halfway vector and the normal ends up being comparable to the relationship between the reflection vector and the camera.

The reason has something to do with modelling the orientation of micro-facets of the surface that would reflect light.

What's interesting is that in many circumstances, Blinn-Phong is actually more empirically accurate.

And even if not, it's hard to visually tell the difference (although there can be some strange behavior when highlights are at extreme angles and at high levels of shininess)

Blinn-Phong comparison



At the same shininess level, Blinn looks less shiny. However, if we quadruple the shininess, it looks very close to Phong, and it's cheaper to compute on older hardware.

Image by Brad Smith, CC-BY-SA 3.0

Nowadays

We usually use per-fragment lighting. Almost all modern games will do this by default unless they are deliberately going for a retro look.

I haven't noticed a huge performance difference between Blinn and regular Phong, but the look might be preferable, especially if going for a retro look.

Anyway, let's take questions, and then see the code…

Questions?

The code

```
fn specular_color(
    light_dir: vec3f,
    light_color: vec3f,
    norm: vec3f,
    eye_dir: vec3f
)-> vec3f
{   // note, `mtl` is a global material value.
    let reflected = reflect(-light_dir, norm);
    let brightness = max(0.0, dot(reflected, eye_dir));
    let shine = pow(brightness, mtl.shininess);
    return light_color * shine * mtl.specular;
}
```

The code (2)

That was specifically how we compute specular. To get the complete Phong color, we compute diffuse and specular for every light, add ambient at the end, and multiply all the terms by their material responses. See the sample for a complete implementation.

We're storing the material in a struct. We'll talk about how to do that soon.

The Blinn version is similar, but we don't perform `reflect` (which is the slower part of the original). Instead, we average the light and camera vectors, and dot the result with the normal…

The code (3)

```
fn specular_color_blinn(
    light_dir: vec3f,
    light_color: vec3f,
    norm: vec3f,
    eye_dir: vec3f
)-> vec3f
{
    let halfway = normalize(light_dir + eye_dir);
    let brightness = max(0.0, dot(halfway, norm));
    let shine = pow(brightness, mtl.shininess);
    return light_color * shine * mtl.specular;
}
```

Structs in WGSL

Notice that we were accessing fields from a global struct variable named `mtl`. `mtl.specular` and `mtl.shininess`. How does that work?

We've already created a struct for our vertex outputs before. But it's possible to have structs be `uniforms` too. We can send them from our Javascript code.

This is convenient for several reasons:

1. It means we can copy over all the material data in one copy.
2. It means we don't have to create 4 different buffers or views, one for each material parameter.
3. It's faster

Structs in WGSL (2)

Let's look at the struct we store our material parameters in:

```
struct Material {
    ambient: vec3f, // bytes 0-11, padding 4
    diffuse: vec3f, // bytes 16-27, padding 4
    specular: vec3f, // bytes 32-43
    shininess: f32,  // bytes 44-47 (where padding would be)
};
@group(0) @binding(2) var<uniform> mtl: Material;
```

Seems like a pretty straightforward struct. It stores our 4 Phong material parameters. But what are those comments about?

Alignment

We can't ignore it anymore. We have to talk about alignment.

This is a rather challenging part of modern graphics programming. It can't easily be helped.

Basically, each struct we want to send as a uniform needs to come from a buffer of some kind.

And the data in that buffer needs to have very specific locations in order to get loaded into the correct fields of the struct.

The rules are very similar to C language alignment rules, so let's briefly talk about those…

Alignment in C

Consider these two structs (as written, without any `pragmas`):

```
typedef struct {
  uint64_t a;
  uint32_t b;
  char c;
} A;
typedef struct {
  char c;
  uint64_t a;
  uint32_t b;
} B;
```

What are their sizes?

Alignment in C (2)

You might add up the bytes and say “13”, because a 64-bit int is 8 bytes, a 32-bit int is 4 bytes, and a char is always 1 byte. 8 + 4 + 1 seems to be 13, right?

Except that’s not what we get:

```
printf("%d %d\n", sizeof(A), sizeof(B));
```

will print 16 and 24.

This immediately raises two questions:

1. Why isn’t either of them 13?
2. Why are they *different*?

Alignment in C (3)

The answer to both questions boils down to **aligment**.

Alignment is the idea that data can only live at addresses of a particular multiple. For example, a 4-byte-aligned int could be at address 20, because 20 is a multiple of 4, but it could not be at 21, 22, or 23.

The alignment of a datatype is a number, describing the valid multiples.

Modern x86 CPUs are actually uncommonly forgiving of mis-aligned data, so you may not have really thought about it too much if you're on that platform, but other platforms, and especially GPUs care about this.

Why?

Alignment (2)

There are a couple of reasons: some memory hardware may literally just not be byte-addressible, and even if it is, there are performance benefits to being aligned.

Also consider the fact that any practical system will be cached.

RAM latency is so long compared to a CPU/GPU clock, that not having cache is simply not an option. You have to have it. Your computer would be orders of magnitude slower if you didn't have it.

Cache is broken into pages, which are further broken into lines.

Let's say a cache line is 32-bytes for sake of argument.

Alignment (3)

Suppose we want 3 vectors and one 32-bit int:

- Vector 1: 4 floats, 4 bytes each, 16-bytes (going from 0 to 15)
- Vector 2 starts at byte 16 and goes to 31
- Vector 3 starts at byte 32 and goes to 47
- The int starts at 48, and goes up to 51

This means that vectors 1 and 2 are cleanly in the first cache line, and vector 3 and the int are in the second.

No problem.

What happens if we move the order?

Alignment (4)

Without alignment, we could end up with something like this:

- The int (from 0 to 3)
- Vector 1 (from 4 to 19)
- Vector 2 (from 20 to 35)
- Vector 3 (from 36 to 51)

This means that vector 2 is partly in cache line 1 and partly in line 2.

This has a couple of effects:

- Loading it is at least twice as expensive (maybe more)
- It increases the chance of eviction (both values need to be there)

Alignment (5)

So far, this seems rather minor. “Okay, why can’t it just be slower if I do it wrong?”

Well, because modern GPUs typically have hardware that is designed around addressing giant chunks.

For example, the *minimum* index size of a modern NVidia GPU will be 32-byte blocks. But you may encounter 64 or even 128-byte transfers.

This is just how the underlying RAM works (e.g., GDDR5). The electronics are simpler if it doesn’t have to be byte addressible.

Alignment (6)

As a result, if one of those integers messes up the alignment of an entire buffer, it could literally halve the memory bandwidth.

As a result, WebGPU just will not let you have unaligned data.

In fact, the alignment is hard coded for different types.

First, let's see what it is for vectors and scalars…

WGSL alignments

There are basically 3 kinds of scalar for uniforms: `i32`, `u32`, and `f32`[1]

All of them are 4-bytes wide, and all have 4-byte alignment.

That means, the address they live at must be divisible by 4.
I.e., `address % 4 == 0`.

Vector alignment depends on the number of elements:

- A `vec2` has a size of 8-bytes, and an alignment of 8-bytes
- A `vec3` has a size of 12-bytes, **but an alignment of 16-bytes**
- A `vec4` has a size of 16-bytes, and an alignment of 16-bytes

[1] technically there are also `i16`, `u16`, and `f16`, but you can't put them in uniforms.

WGSL alignments (2)

The basic rule for scalars and vectors is that their alignment will be their size rounded up to the next power of 2. So a vec3 goes to 16.

Therefore, suppose you want to send 3 vectors in a struct. You need to first put them in a buffer:

```
// wgsl:
struct something { v1: vec3f, v2: vec3f, v3: vec3f };
@group(...) binding(...) data: something;

device.queue.writeBuffer(buf, 0, new Float32(
  [x1, y1, z1, 0, x2, y2, z2, 0, x3, y3, z3, 0]);
```

WGSL alignments (3)

Notice that when we load the data, we had to insert those zeros between vectors. The reason is alignment. They are called **padding**.

Those padding zeros are ignored. They could be anything.

If you don't put them in, but did something like this:
`[x1, y1, z1, x2, y2, ...]`

That `x2` would end up being in the padding area between vectors. It would be lost, and you would end up loading `y2` into the `x2` spot of the field in the struct. The data would be wrong starting from that point.

WGSL alignments (4)

So, here is our material struct:

```
struct Material {
    ambient: vec3f, // bytes 0-11, padding 4
    diffuse: vec3f, // bytes 16-27, padding 4
    specular: vec3f, // bytes 32-43
    shininess: f32,  // bytes 44-47 (in specular's padding)
};
```

The first field always starts at offset 0, because 0 is always an aligned address (it's a multiple of anything).

The second field starts at 16, because the previous field ends at 12, and that's not aligned for 3-vectors. We go to the next aligned address: 16.

WGSL alignment (5)

We put specular at 32 for the same reason.

However, specular has a size of 12-bytes. So the next address, 44, is actually aligned for the f32 we want to store (shininess).

Does this make sense? It was confusing to me when I first saw it in C, and it's about to get a little moreso...

Questions?

WGSL alignment of matrices and arrays

Now let's briefly cover more complex types.

Matrices are weird: they are considered like collections of vectors, but their size is increased so they have no padding.

That is, a 3-by-3 matrix, is treaded like 3 vector-3s, but with no padding at the end.

3 vector-3s would normally require 44 bytes like we saw before, and there would be 4 bytes of padding.

The 3-by-3 matrix doesn't have that padding. So it's 48 bytes. The rule for matrices: multiply the number of basis vectors by their alignment.

WGSL alignment of matrices and arrays (2)

Arrays have their own rule: their alignment is the same as the alignment of their element type. So an array of vec3s has an alignment of 16.

The size of an array is the sum of the sizes of its elements, but rounded up to remove the padding at the end. So an array of 3 vec3s would have a size of 48-bytes, just like the matrix 3 example.

Here is a table with every type's size and alignment.

Let's load materials

To load our material, we have a function called `packMaterial` which generates a `Float32Array` that we can copy to the uniform's buffer:

```
function packMaterial(
    ambient: vec3,
    diffuse: vec3,
    specular: vec3,
    shininess: number
): Float32Array
{ // remember that the ... operator copies over the elements of the array
    return new Float32Array([
        ...ambient, 0, // add the 0 for padding
        ...diffuse, 0,
        ...specular, // whoops, no padding here, shininess goes there
        shininess
    ]);
}
```

Let's load materials (2)

Once the material is packed, we can copy it into a buffer:

```
this.device.queue.writeBuffer(
  matBuffer, 0, packMaterial(...));
```

This buffer needs to be whichever buffer is linked to the bind group for the object we're going to draw later. In my sample, there is only one.

After this copy, the fields of the `mtl` variable in the shader will be correct. We overwrote the fields of the global struct with the correct alignment.

Questions?

Kinds of lights

So far we've been talking exclusively about directional lights.

Directional lights are defined by a direction vector. They don't have a position in space.

That might seem strange, but directional lights represent lights that are so bright and so far away, that moving to the left and right will not meaningfully move the vector, so the vector can be a constant.

[Can anyone think of an everyday example of a light that is so bright and so far away that moving doesn't affect the angle that it shines on you?]

Kinds of lights (2)

Yeah, the sun would be a directional light in most scenes.

And actually, it's kind of the only one we normally think about. The moon is too, but it doesn't cast very much light.

Ultimately, the directional lights are up to us. We could decide to make a normal lamp a directional light if our scene is super tiny.

And the sun would *not* be a directional light in a solar-system simulation, since then its direction would change relative to the planets frequently.

So, it would be too limiting if that were the only kind of light.

Point lights

So we're also going to learn about **point lights**.

A point light has a position in space. It is considered infinitely small.

Point lights are a bit different from directional lights for two reasons:
1. The direction to them is different for every distinct vertex position.
2. They **attenuate** (i.e., get dimmer as you get farther away)

The first difference isn't too hard to deal with. Just subtract the position of the vertex from the position of the light. Normalize, and that's our light direction.

So let's talk about **attenuation**...

Attenuation

Attenuation is how much the brightness falls off based on distance.

Previously, we took the direction to the light and used the cosine with the normal to get its diffuse brightness, and the cosine between the reflection and camera (or halfway and normal) to get the specular brightness.

We're still going to do that, but now we're going to *multiply the result by the amount of attenuation.*

The farther away, the smaller the attenuation factor will get. This will make the light get dimmer as it gets far away.

Attenuation (2)

In reality, light sources follow an inverse squares law.

That means that the light cast on an object is proportional to $\frac{1}{d^2}$

So we can just eyeball a good value k_q, so that $\frac{k_q}{d^2}$ looks good.[1]

Unfortunatelly, you might be searching a long time, because for most scenes, this will be way too dark.

For example, if the light has a color (0.5, 0.5, 0.5) and we choose 1 as our constant, we get 1/100 brightness only 10 units away.

[1]The "q" stands for quadratic. We'll see why very soon.

Attenuation (3)

The problem is that there is a key difference between real life and the simple scenes that we are going to be developing:

Real life has RTX (i.e., global illumination).

As a result, the light isn't just coming from the light source, it's also bouncing off all the surfaces in the room.

So yes, the brightness decays with the square of distance, but the light is mostly conserved by other surfaces in the room, which makes up for it.

But in our scenes, we *don't* have global illumination, so quadratic attenuation gets dark way too fast…

Attenuation (4)

You might think, “so, just make the constant really small!”

And that seems to work. If set the quadratic constant of attentuation to, say, 0.1, then, at a distance of 10, we end up with 1 / .1 / 100 = .1. Not too bad!

Unfortunately, with a low constant like this, we end up making the light *too bright* when we’re close. If we’re 1 unit away, the attenuation factor is times 10. The light is 10-times brighter than its base brightness.

So let’s try adding a linear term. Let’s call it k_l

Attenuation (5)

So suppose we choose a linear attenuation term of 1

That means, at a distance of 1 the light is its standard brightness.

At a distance of 2 it is half as bright. At 3 it is one-third as bright.

The falloff works okay, but it's a little drab, and it makes even small lights cast a very long distance.

So we typically use *both* terms: some linear and some quadratic.

But there's one more problem…

Attenuation (6)

What if we're really close to the light?

Regardless of whether we use linear or quadratic attenuation, we'll end up dividing by a tiny number. The light will blow out the colors of vertices close to it.

To fix this, we add a constant: k_c.

This is added to the linear and quadratic attenuation terms, and it can guarantee that the amount of the attenuation will be greater than zero, preventing the blow-up.

The attenuation formula

Therefore, once we compute our Phong brightness, multiply by:

$$\alpha = \left(k_c + k_l d + k_q d^2\right)^{-1}$$

This will reduce (or increase in some cases) the color of the light.

This is something we only do for point lights. Directional lights are seen as being infinitely far away, but being so bright that they don't have attenuation.

Attenuation WGSL

```
fn attenuate(
    input_color: vec3f,
    distance: f32,
    factors: vec3f,
) -> vec3f
{ // constant in factors.x, linear in factors.y, etc.
    return input_color / (
        factors.x +
        factors.y * distance +
        factors.z * distance * distance
    );
}
```

Let’s look at the code

I’ve implemented attenuation in the sample. It’s the little red light.

Let’s play with the sample, and then look at the code to see how it works.

Questions?

Swizzling

One last little thing I haven't covered.

GPUs are designed to work with vectors. The basic registers and opcodes are specifically designed for that kind of data.

Imagine how inconvenient it would be if we needed a special instruction for every channel of the vector. For example, if multiplying by the `.x` component had a different instruction to multiplying by the `.y` component.

We'd need at least 4 versions of every operation, not even accounting for combinations like "multiply x by y"...

Swizzling (2)

As a result, the low-level assembly languages in GPUs have flexible operand formats: you can specify which components you want an operation to apply to.

This feature is exposed in WGSL (and most other shading languages) through a feature called **swizzling**.

Swizzling is when the fields of vectors are repeated or reordered.

For example, if `v` is a vector, `v.x` is its `x` component. But we can also say `v.xx`, to create a 2-dimensional vector that has `v`'s `x` component as both its `x` and `y`.

Swizzling (3)

I store my light positions as 4-d vectors, so that if the w component is zero, it's treated as a directional vector.

But, when it's *not* directional, I don't want to include the w component in the distance calculation. It would be wrong. So I *downcast* it like this:

```
let offset = original.xyz
```

That `.xyz` is swizzling. I'm creating a new 3-vector from the `x`, `y`, and `z` components of the original vector.

But I don't even have to keep them in the same order…

Swizzling (4)

Recall that the cross product is defined like this:

$$a \times b = \left(a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x\right)$$

We can use swizzling to really optimize this:

```
let result = a.yzx * b.zxy - a.zxy * b.yzx
```

Those * operations are pairwise multiplications. Multiply a's x by b's x, a's y by b's y, etc.

Pretty cool, huh?

Homework and Reading

One last thing before we go. Back before shaders, there were three kinds of lights that GPUs had built-in support for:

1. Directional lights
2. Point lights
3. Spotlights

We haven't talked about that third one…

We're going to see it in project 5, but for now, how do you think you would make spotlights work?

Hint: they have a position but also a direction they're pointing…

Homework and Reading (2)

I strongly recommend reading this article on alignment.

Be sure to read this book chapter if you haven't.

Questions?